

Linguaggi di programmazione

Leonardo Ganzaroli

Indice

1	Macchine astratte	2
2	Descrizione di linguaggi	6
2.1	Analisi lessicale	7
2.2	Analisi sintattica e semantica	8
3	Nomi e ambiente	9
4	Gestione della memoria	11
5	Meccanismi di controllo	12
6	Funzioni	14

1 Macchine astratte

Definizione Dato un linguaggio di programmazione L . Una macchina astratta M_L per L è un qualsiasi insieme di strutture dati e di algoritmi che permettano di memorizzare ed eseguire programmi scritti in L .

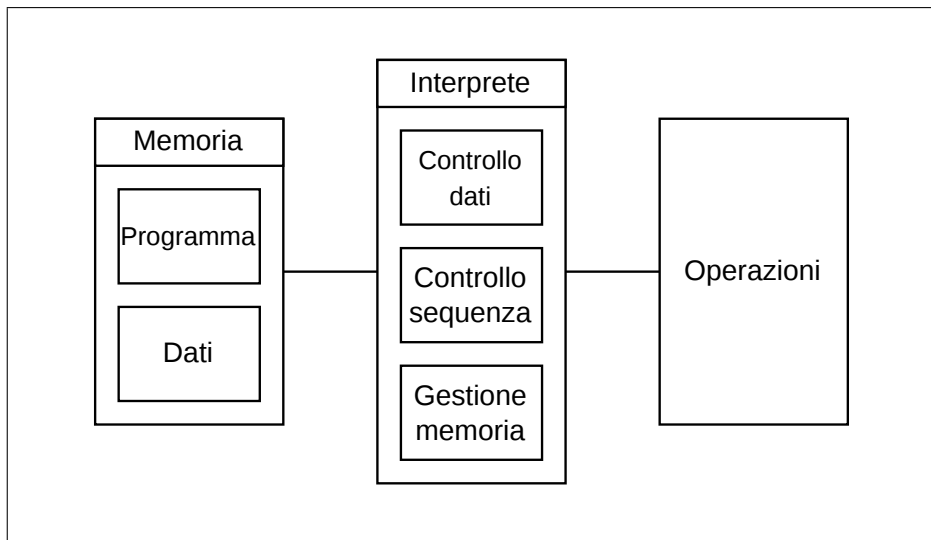


Figura 1: Struttura generale di una m.a.

Indipendentemente dal linguaggio ogni interprete ha un flusso di esecuzione e delle operazioni di base comuni:

- Operazioni per l'elaborazione dei dati primitivi
- Op. e strutture dati per il controllo di sequenza
- Op. e s.d. per il trasferimento dei dati
- Op. e s.d. per la gestione della memoria

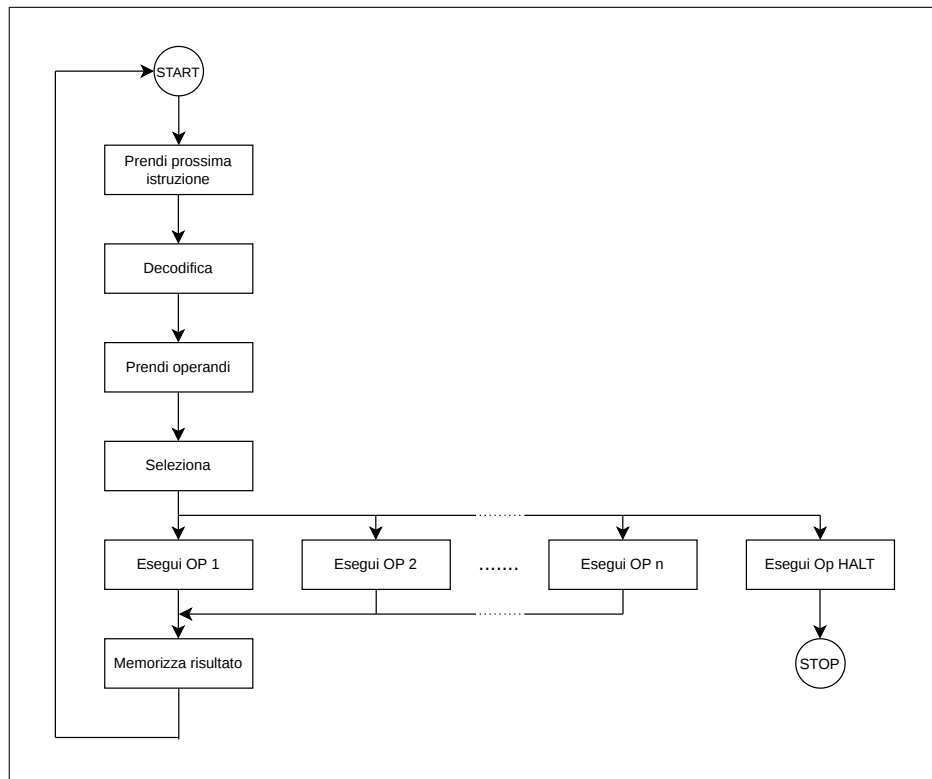


Figura 2: Ciclo di esecuzione

Definizione Data una m.a. M_L . Il linguaggio L “compreso” dall’interprete della macchina è detto linguaggio macchina di M_L .

Per andare effettivamente a realizzare un m.a. ci sono 3 strade:

- Costruirla fisicamente
- Simularla tramite un altro linguaggio (la cui m.a. è già implementata)
- Combinare entrambi i metodi

Il secondo metodo presenta a sua volta 3 strade:

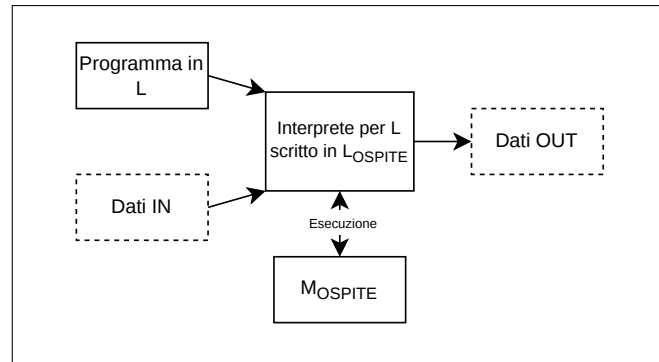


Figura 3: Interpretazione pura

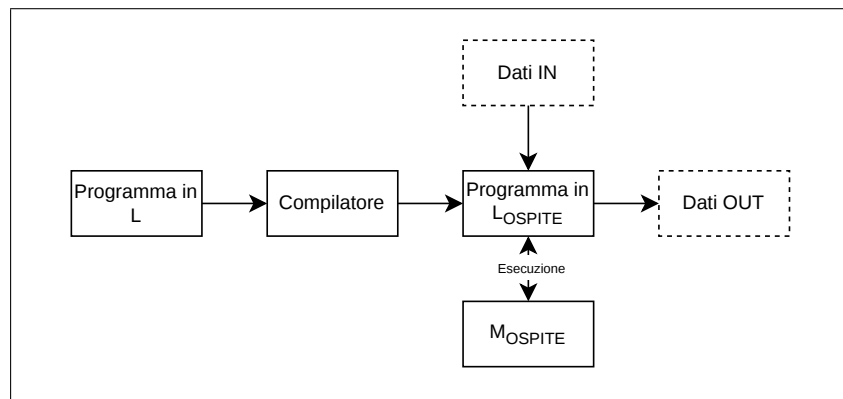


Figura 4: Compilazione pura

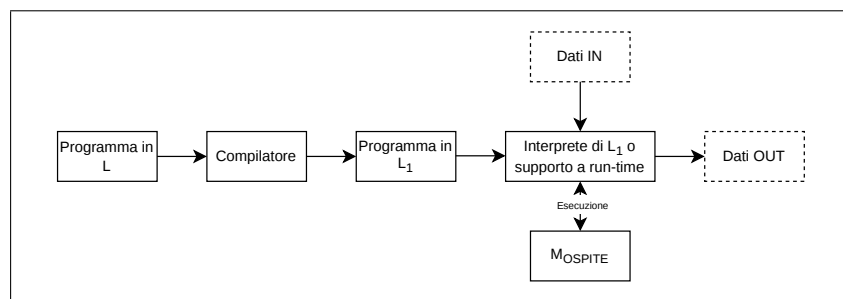


Figura 5: Via di mezzo (quella più usata nella realtà)

In quest'ultimo caso esiste un intero spettro di implementazioni che vanno da quella in cui tutto è simulato a quella in cui tutto è tradotto. La scelta su cosa simulare e cosa tradurre dipende dal linguaggio e dalla macchina ospite, ma in linea di principio i costrutti del linguaggio (da implementare) “lontani” dal linguaggio della macchina ospite si simulano e il resto si compila.

L'uso di queste macchine intermedie permette di creare una gerarchia di m.a. simile allo stack ISO/OSI.

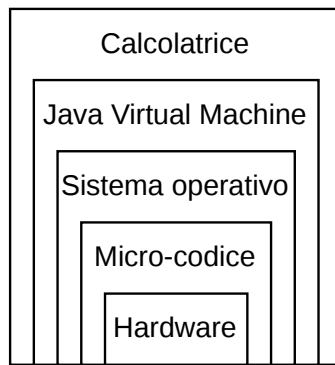


Figura 6: Esempio di gerarchia

2 Descrizione di linguaggi

Un linguaggio di programmazione può essere descritto su 4 livelli:

1. **Grammatica**

- (a) **Lessico**

Quali sequenze di caratteri formano le parole del linguaggio?

- (b) **Sintassi**

Quali sequenze di parole formano frasi corrette?

2. **Semantica**

Che significato ha una certa frase corretta?

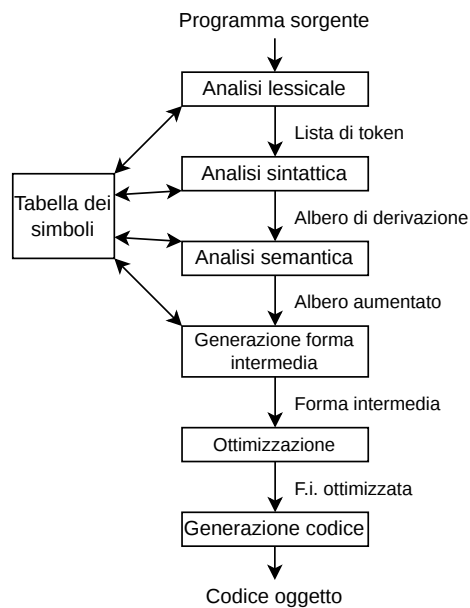
3. **Pragmatica**

Come si usa una frase corretta in modo sensato?

4. **Implementazione**

Come va eseguita una frase corretta in modo da rispettarne la semantica?

Basandosi su questi concetti si può schematizzare il funzionamento del compilatore:



2.1 Analisi lessicale

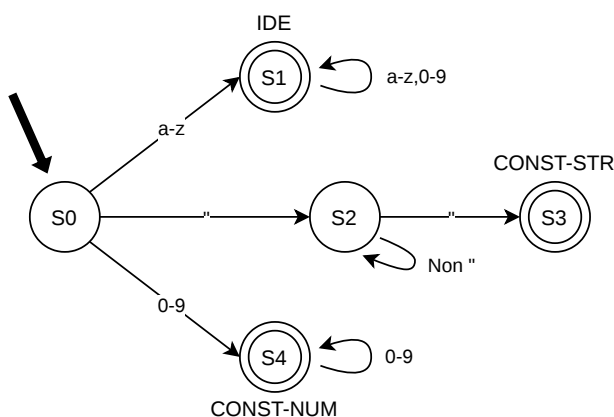
Lo scopo generale dell'analisi lessicale è quello di riconoscere nella stringa di ingresso dei gruppi di caratteri corrispondenti a determinate categorie.

Token	Coppia $\langle \text{nome}, \text{valore} \rangle$	$\langle IDE, x1 \rangle$
Nome	Identifica classe di token	<i>IDE</i>
Valore ¹	Identifica specifico token	<i>x1</i>
Pattern	Descrizione forma valore	$(x y)(x y 0 1)^*$
Lessema	Istanza di un pattern	<i>x1</i>

Dato che la struttura si esprime sotto-forma di espressione regolare il metodo più naturale per effettuare il riconoscimento è usare un automa. Per esempio nella stringa *if (x == 0) printf("zero")* alcuni token possibili sono:

- $\langle CONST - NUM, 0 \rangle$
- $\langle CONST - STR, zero \rangle$
- $\langle IDE, x \rangle$
- $\langle OP, == \rangle$
- $\langle if \rangle$
- $\langle (\rangle$
- $\langle \rangle \rangle$

Dove un possibile automa per le prime 3 categorie è:



¹Se nome e valore coincidono spesso quest'ultimo viene omissso.

2.2 Analisi sintattica e semantica

Il passo successivo consiste nel verificare che la sequenza di token data dal punto precedente costituisca una frase formulata correttamente. Per capirlo si definisce un'opportuna grammatica acontestuale e si prova a creare un albero di derivazione, nel caso non sia possibile la frase è mal strutturata.

$$\begin{aligned} E &\longrightarrow E \text{ OP } T \mid T & \text{OP} &= \{+, -\} \\ T &\longrightarrow T \text{ OP2 } A \mid A & \text{OP2} &= \{*, /\} \\ A &\longrightarrow \text{NUM} \mid (E) \end{aligned}$$

Per $2 + 3 * 5$:

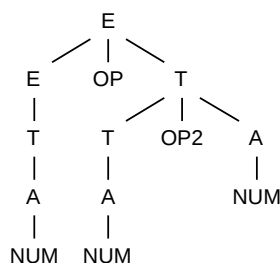


Figura 7: Esempio di costruzione *top-down*

È anche possibile usare l'approccio *bottom-up*, ossia creare l'albero partendo dalle foglie e andando a ritroso.

La correttezza di una frase potrebbe però dipendere anche dal contesto in cui essa si trova, per esempio:

- In Java una variabile locale di una funzione non si può usare prima che gli venga assegnato un valore;
- In Pascal la variabile di controllo usata in un ciclo *for* non si può modificare nello stesso;

Questi tipi di vincoli sono detti “vincoli sintattici contestuali”². L'analisi semantica va a verificare se questi vengano rispettati e facendolo “aumenta” man mano l'albero inserendoci informazioni aggiuntive. Facendo una visita dell'albero finale ottenuto si può già ottenere una prima versione del codice (la forma intermedia).

²Spesso vengono chiamati “vincoli di semantica statica”, intesi come vincoli verificabili staticamente sul testo del programma. Analogamente esistono quelli di “semantica dinamica” attinenti al momento di esecuzione del programma.

3 Nomi e ambiente

Definizione Un nome è una sequenza di caratteri usata per denotare un oggetto³.

Definizione L'insieme delle associazioni nomi-oggetti esistenti a run-time in un certo punto del programma ed in un certo momento dell'esecuzione è detto ambiente.

Definizione Una dichiarazione è un costrutto che permette di introdurre un'associazione nell'ambiente.

Definizione L'aliasing è la situazione in cui un oggetto è associato a più nomi diversi nello stesso ambiente⁴.

Definizione Un blocco è una regione testuale di un programma delimitata da un inizio ed una fine, si dividono in blocchi di procedure/funzioni e blocchi inline.

All'entrata/uscita in/da un blocco avviene un cambiamento di ambiente, rispetto al blocco si distinguono:

- **Ambiente locale**
Associazioni dichiarate localmente.
- **Ambiente non locale**
Associazioni visibili nel blocco ma non dichiarate localmente.
- **Ambiente globale**
Associazioni create all'inizio dell'esecuzione.

Per quanto riguarda quello che succede durante il cambio d'ambiente:

- **In entrata**
 - Creazione di associazioni locali
 - Disattivazione di associazioni non locali il cui nome viene ridefinito localmente
- **In uscita**
 - Distruzione di associazioni locali
 - Riattivazione di associazioni disattivate

³Un oggetto può essere: una variabile, un parametro, una procedura, una costante, un tipo, ecc...

⁴Un esempio è la *union* in C, dove variabili diverse denotano la stessa zona di memoria.

A livello più generale le operazioni eseguibili su nomi e ambiente sono:

- Creazione di un'associazione
- Disattivazione di un'associazione
- Riattivazione di un'associazione
- Distruzione di un'associazione
- Riferimento ad un oggetto tramite nome

Invece per gli oggetti:

- Creazione⁵
- Distruzione
- Accesso
- Modifica

Sorge però un problema di visibilità nell'ambiente non locale, per esempio nel seguente codice la x usata dentro $p()$ a quale x esterna si riferisce?

```
A
|  int  $x \leftarrow 0$ ;
|  procedure  $p()$ 
|  |   $x \leftarrow 1$ ;
|  B
|  |  int  $x$ ;
|  |   $p()$ ;
|  |  write( $x$ );
```

Per evitare casi di questo tipo va usata una regola di scope (visibilità), ce ne sono 2:

- **Statico**

Si cerca nel blocco superiore, se non c'è in quello si continua a salire.

(x di **A**, alla fine viene stampato 1)

- **Dinamico**

Si usa l'occorrenza più recente da un punto di vista temporale.

(x di **B**, alla fine viene stampato 0 perché la x di B sparisce uscendo dal blocco)

⁵In alcuni linguaggi la creazione di un oggetto, di un nome e l'associazione tra i 2 è simultanea.

4 Gestione della memoria

Definizione Un record di attivazione (o frame) è uno spazio in memoria allocato per una procedura o in generale per un blocco.

Una m.a. può gestire la memoria in 3 modi:

1. Staticamente

Prima dell'esecuzione il compilatore va a designare un'area di memoria apposita per ogni oggetto. Questo metodo è perfetto per le variabili locali, le costanti e le singole istruzioni del codice oggetto. Nel caso in cui il linguaggio non supporti la ricorsione si può estendere il metodo anche alle procedure salvando in memoria le loro informazioni locali⁶.

Procedura 1
Info di sistema
Indirizzo di ritorno
Parametri
Variabili locali
Risultati intermedi

2. Dinamicamente

(a) Con pila

L'ingresso in un blocco comporta l'inserimento di un RdA dentro una pila (poi rimosso all'uscita) che viene collegato a quello immediatamente precedente tramite puntatore.

In-line	Procedura
	Puntatore catena statica
	Puntatore catena dinamica
	Variabili locali
	Risultati intermedi
	Parametri
	Indirizzo di ritorno
	Indirizzo del risultato

(b) Con heap

Se il linguaggio prevede comandi per l'allocazione esplicita di memoria si definisce una zona specifica dove è possibile riservare blocchi di dimensioni fisse/variabili.

⁶Chiamate diverse alla stessa funzione fanno riferimento alla stessa zona di memoria.

5 Meccanismi di controllo

Definizione Un'espressione è un'entità sintattica la cui valutazione produce un valore oppure non termina. Generalmente è composta da una singola entità o da un operatore applicato a degli operandi.

Per il secondo caso esistono diversi metodi di rappresentazione:

- **Notazione infissa**

$$(a + b) * (c + d)$$

- **Notazione prefissa**⁷

$$* (+ (a \ b) + (c \ d))$$

- **Notazione postfissa**⁸

$$((a \ b) + (c \ d)) + *$$

(Nelle ultime 2 si possono omettere le parentesi se si conosce l'arietà delle operazioni)

Cambiando la notazione cambia anche il metodo di valutazione, rispettivamente:

- Bisogna definire delle regole di precedenza, altrimenti serve un uso massiccio di parentesi;
- Scansione da sinistra a destra ed uso di una pila;
- Come il precedente.

Ogni notazione si può ottenere con un tipo di visita diversa dell'albero di derivazione (considerando solo i simboli terminali), per esempio nella figura 7:

- **In ordine**⁹

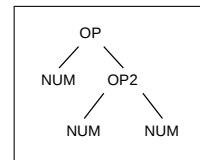
$$\text{NUM OP NUM OP2 NUM} \rightarrow 2 + 3 * 5$$

- **Pre-ordine**¹⁰

$$\text{OP NUM OP2 NUM NUM} \rightarrow + 2 * 3 5$$

- **Post-ordine**¹¹

$$\text{NUM NUM NUM OP2 OP} \rightarrow 2 \ 3 \ 5 \ * \ +$$



⁷ Anche detta notazione polacca.

⁸ Anche detta notazione polacca inversa.

⁹ Sx, radice, dx.

¹⁰ Radice, sx, dx.

¹¹ Sx, dx, radice.

Definizione Un effetto collaterale è un'azione che influenza i risultati di una computazione senza restituire esplicitamente un valore.

Definizione Un comando è una entità sintattica la cui valutazione non necessariamente restituisce un valore, ma può avere un effetto collaterale.

Definizione Una variabile modificabile è una locazione di memoria (con nome) contenente dei valori che possono cambiare nel tempo.

Alcuni comandi:

- Assegnamento
- **Controllo di sequenza**
 - ;
 - Begin...End
 - GOTO
 - Continue
 - Break
 - Return
- **Condizionali**
 - If (e sue varianti)
 - Switch...Case...Default
- **Iterativi**
 - Indeterminati
 - * While
 - * Do...While
 - * Repeat
 - * Until
 - Determinati
 - * For
 - * For-each

6 Funzioni

Definizione Una funzione è una porzione di codice identificata da un nome¹², dotata di ambiente locale proprio e capace di scambiare informazioni col resto del codice mediante parametri¹³, valore di ritorno e ambiente non locale.

Esistono diversi metodi di funzionamento dei parametri:

- **Per valore**

I parametri formali copiano i valori dei parametri attuali.

- **Per riferimento**

I parametri formali diventano degli alias dei parametri attuali.

- **Per costante**

Come il precedente ma *read-only*.

- **Per risultato**

Come quello per valore ma in uscita.

- **Per valore-risultato**

Valore + risultato.

- **Per nome**

Le occorrenze dei parametri formali vengono sostituite con quelli attuali, per esempio in:

```
int x ← 2;
foo(name int y)
└─ return y + y;
a ← foo(x ++);
```

$y+y$ diventa $(x++)+(x++)$ dove $x++$ viene valutato ad ogni occorrenza, quindi:

1. Il primo operando è 2
2. x diventa 3
3. Il secondo operando è 3
4. x diventa 4
5. a è 5

¹²Il nome della funzione appartiene all'ambiente non locale

¹³Si distinguono quelli che appaiono nella definizione (*formali*) e quelli che appaiono nella chiamata (*attuali*).